

REASONING_MODELS

Reasoning Models Guide

This guide explains how to use reasoning models in HelixCode for advanced problem-solving and analysis tasks.

Table of Contents

- [Overview](#)
- [Supported Models](#)
- [Configuration](#)
- [Usage Examples](#)
- [Thinking Budget Recommendations](#)
- [Cost Considerations](#)
- [Best Practices](#)
- [API Reference](#)

Overview

Reasoning models are specialized LLMs that perform explicit step-by-step thinking before producing their final output. Unlike standard models, reasoning models:

- Show their internal thought process
- Break down complex problems into steps
- Verify their reasoning before providing answers
- Can self-correct during the reasoning process

HelixCode provides comprehensive support for reasoning models, including:

- Automatic thinking extraction and separation
- Token budget management for thinking tokens
- Cost calculation for reasoning operations
- Model-specific prompt formatting
- Configurable reasoning effort levels

Supported Models

OpenAI o-series

OpenAI o1, o1-preview, o1-mini - Provider: OpenAI - Thinking Tags: <thinking>
- Default Budget: 10,000 tokens - Pricing: \$15/1M input, \$60/1M output - Best For: Complex reasoning, mathematics, coding problems

OpenAI o3 (Future) - Provider: OpenAI - Estimated Budget: 10,000 tokens - Estimated Pricing: \$20/1M input, \$80/1M output - Best For: Advanced reasoning tasks

OpenAI o4 (Future) - Provider: OpenAI - Estimated Budget: 10,000 tokens - Estimated Pricing: \$25/1M input, \$100/1M output - Best For: Highly complex reasoning

Claude Models

Claude Opus (Extended Thinking) - Provider: Anthropic - Thinking Tags: <thinking> - Default Budget: 5,000 tokens - Pricing: \$15/1M input, \$75/1M output - Best For: Deep analysis, research, comprehensive problem-solving

Claude Sonnet (Extended Thinking) - Provider: Anthropic - Thinking Tags: <thinking> - Default Budget: 5,000 tokens - Pricing: \$3/1M input, \$15/1M output - Best For: Balanced reasoning and cost-effectiveness

DeepSeek Models

DeepSeek R1 - Provider: DeepSeek - Thinking Tags: <think> - Default Budget: 8,000 tokens - Pricing: \$2.19/1M input, \$8.19/1M output - Best For: Cost-effective reasoning, research applications

DeepSeek Reasoner - Provider: DeepSeek - Thinking Tags: <think> - Default Budget: 8,000 tokens - Pricing: \$2.19/1M input, \$8.19/1M output - Best For: Specialized reasoning tasks

QwQ Models

QwQ-32B - Provider: Alibaba Cloud - Thinking Tags: <thinking> - Default Budget: 7,000 tokens - Pricing: \$0.50/1M input, \$1.50/1M output (varies by provider) - Best For: Budget-conscious reasoning applications

Configuration

Basic Configuration

```
import "dev.helix.code/HelixCode/internal/llm"

// Create a default reasoning config
config := llm.DefaultReasoningConfig()
config.Enabled = true
config.ThinkingBudget = 5000
config.ReasoningEffort = "medium"
```

Model-Specific Configuration

```
// OpenAI o1 configuration
o1Config := llm.NewReasoningConfig(llm.ReasoningModelOpenAI_O1)

// Claude Opus configuration
claudeConfig :=
    llm.NewReasoningConfig(llm.ReasoningModelClaude_Opus)

// DeepSeek R1 configuration
deepseekConfig :=
    llm.NewReasoningConfig(llm.ReasoningModelDeepSeek_R1)

// QwQ-32B configuration
qwqConfig := llm.NewReasoningConfig(llm.ReasoningModelQwQ_32B)
```

Configuration Options

Option	Type	Description	Default
Enabled	bool	Enable reasoning mode	false
ExtractThinking	bool	Extract thinking blocks	true
HideFromUser	bool	Hide thinking from end users	false
ThinkingTags	string	XML tags for thinking blocks	“thinking”
ThinkingBudget	int	Token budget for thinking (0=unlimited)	0

Option	Type	Description	Default
ReasoningEffort	string	Effort level: “low”, “medium”, “high”	“medium”
ModelType	ReasoningModelType	Specific model type	generic

Usage Examples

Example 1: Basic Reasoning Request

```
package main

import (
    "context"
    "fmt"
    "dev.helix.code/HelixCode/internal/llm"
)

func main() {
    // Create reasoning config for Claude Opus
    config :=
        llm.NewReasoningConfig(llm.ReasoningModelClaude_Opus)

    // Format prompt for reasoning
    prompt := "Solve the equation: 2x + 5 = 15"
    formattedPrompt := llm.FormatReasoningPrompt(prompt, config)

    // Make LLM request (pseudo-code)
    response := makeRequest(formattedPrompt)

    // Extract reasoning trace
    trace := llm.ExtractReasoningTrace(response, config)

    fmt.Println("Thinking Process:")
    for i, thinking := range trace.ThinkingContent {
        fmt.Printf("Step %d: %s\n", i+1, thinking)
    }

    fmt.Println("\nFinal Answer:")
    fmt.Println(trace.OutputContent)

    fmt.Printf("\nToken Usage: %d thinking + %d output = %d\n",
        total\n",
```

```

        trace.ThinkingTokens, trace.OutputTokens,
        trace.TotalTokens)
    }

```

Example 2: Budget Management

```

// Get recommended budget for use case
budget := llm.GetReasoningBudgetRecommendation("complex")

// Create config with budget
config := llm.NewReasoningConfig(llm.ReasoningModelOpenAI_01)
config.ThinkingBudget = budget

// Apply budget enforcement
trace := llm.ExtractReasoningTrace(response, config)
budgetedTrace := llm.ApplyReasoningBudget(trace, config)

if budgetedTrace.ThinkingTokens < trace.ThinkingTokens {
    fmt.Println("Warning: Thinking was truncated to fit budget")
}

```

Example 3: Cost Calculation

```

// Calculate cost for reasoning operation
trace := llm.ExtractReasoningTrace(response, config)
thinkingCost, outputCost, totalCost :=
    llm.CalculateReasoningCost(
        trace,
        config,
        llm.ReasoningModelClaude_Sonnet,
    )

fmt.Printf("Cost Breakdown:\n")
fmt.Printf("  Thinking:  %.6f\n", thinkingCost)
fmt.Printf("  Output:    %.6f\n", outputCost)
fmt.Printf("  Total:     %.6f\n", totalCost)

```

Example 4: Model Detection

```

// Automatically detect reasoning models
modelName := "o1-preview"
isReasoning, modelType := llm.IsReasoningModel(modelName)

if isReasoning {
    config := llm.NewReasoningConfig(modelType)
}

```

```

    fmt.Printf("Using reasoning model: %s\n", modelType)
} else {
    fmt.Println("Using standard model")
}

```

Example 5: Custom Thinking Tags

```

// DeepSeek uses <think> tags
config := llm.DefaultReasoningConfig()
config.Enabled = true
config.ThinkingTags = "think"

// Support multiple tag types
config.ThinkingTags = "thinking,think,internal_thoughts"

trace := llm.ExtractReasoningTrace(response, config)

```

Example 6: Effort Levels

```

// Low effort - quick analysis
configLow :=
    llm.NewReasoningConfig(llm.ReasoningModelClaude_Sonnet)
configLow.ReasoningEffort = "low"

// High effort - comprehensive analysis
configHigh :=
    llm.NewReasoningConfig(llm.ReasoningModelClaude_Opus)
configHigh.ReasoningEffort = "high"

// Optimize config automatically
optimized := llm.OptimizeReasoningConfig(configHigh,
    context.Background())

```

Example 7: Config Merging

```

// Base configuration
baseConfig := llm.DefaultReasoningConfig()
baseConfig.Enabled = true
baseConfig.ThinkingBudget = 5000

// User overrides
userConfig := &llm.ReasoningConfig{
    ReasoningEffort: "high",
    ThinkingBudget: 10000,
}

```

```
// Merge configurations
finalConfig := llm.MergeReasoningConfigs(baseConfig, userConfig)
```

Thinking Budget Recommendations

Choose an appropriate thinking budget based on your use case:

Use Case	Recommended Budget	Description
Simple	2,000 tokens	Quick queries, basic calculations
Standard	5,000 tokens	General reasoning, moderate complexity
Complex	10,000 tokens	Advanced problems, multi-step reasoning
Research	20,000 tokens	Deep analysis, comprehensive research

Budget Guidelines

- **Low Effort:** 3,000 tokens - Brief analysis
- **Medium Effort:** 7,000 tokens - Thorough analysis
- **High Effort:** 15,000 tokens - Comprehensive, detailed analysis

Budget Considerations

1. **Model Capabilities:** Different models use thinking tokens differently
2. **Task Complexity:** More complex tasks require more thinking tokens
3. **Cost Management:** Higher budgets increase costs proportionally
4. **Quality Trade-offs:** Very low budgets may truncate important reasoning

Cost Considerations

Pricing Comparison (per 1M tokens)

Model	Thinking/Input Cost	Output Cost	Cost-Effectiveness
OpenAI o1	\$15	\$60	Moderate
OpenAI o3	\$20 (est.)	\$80 (est.)	Lower
OpenAI o4	\$25 (est.)	\$100 (est.)	Lower

Model	Thinking/Input Cost	Output Cost	Cost-Effectiveness
Claude Opus	\$15	\$75	Moderate
Claude Sonnet	\$3	\$15	High
DeepSeek R1	\$2.19	\$8.19	Very High
QwQ-32B	\$0.50	\$1.50	Extremely High

Cost Examples

Example 1: Simple Query with Claude Sonnet - Thinking: 1,000 tokens → \$0.003
 - Output: 500 tokens → \$0.0075 - **Total: \$0.0105**

Example 2: Complex Analysis with OpenAI o1 - Thinking: 5,000 tokens → \$0.075
 - Output: 2,000 tokens → \$0.12 - **Total: \$0.195**

Example 3: Research Task with DeepSeek R1 - Thinking: 10,000 tokens → \$0.0219
 - Output: 3,000 tokens → \$0.0246 - **Total: \$0.0465**

Cost Optimization Strategies

1. Choose the Right Model

- Use DeepSeek or QwQ for cost-sensitive applications
- Use Claude Sonnet for balanced cost/performance
- Reserve OpenAI o1/Opus for critical tasks

2. Set Appropriate Budgets

- Don't over-provision thinking tokens
- Use recommended budgets as starting points
- Monitor actual usage and adjust

3. Cache When Possible

- Reuse reasoning for similar queries
- Cache expensive reasoning operations
- Consider prompt caching for large contexts

4. Effort Level Management

- Use "low" effort for simple queries
- Reserve "high" effort for complex problems
- Default to "medium" for general use

Best Practices

1. Model Selection

Use OpenAI o1 when: - Solving complex mathematical problems - Working on advanced coding challenges - Requiring high accuracy on reasoning tasks - Budget allows for premium pricing

Use Claude Opus when: - Performing deep analysis or research - Generating comprehensive reports - Needing extensive context understanding - Balance of quality and cost matters

Use Claude Sonnet when: - Cost-effectiveness is important - Standard reasoning complexity - High-volume reasoning operations - Production environments with budget constraints

Use DeepSeek R1 when: - Minimizing costs is critical - Research or experimental applications - High-volume reasoning with budget limits - Acceptable to use newer/less proven models

Use QwQ-32B when: - Extremely cost-sensitive applications - Testing or development environments - Local or self-hosted deployments available - Lower stakes reasoning tasks

2. Thinking Extraction

```
// Always extract and log thinking for debugging
trace := llm.ExtractReasoningTrace(response, config)

// Log thinking for analysis
for i, thinking := range trace.ThinkingContent {
    log.Printf("Reasoning Step %d: %s", i+1, thinking)
}

// Hide thinking from users when appropriate
if config.HideFromUser {
    return trace.OutputContent
}
```

3. Budget Management

```
// Set budgets based on use case
budget := llm.GetReasoningBudgetRecommendation(useCase)
config.ThinkingBudget = budget
```

```
// Always apply budget enforcement
budgetedTrace := llm.ApplyReasoningBudget(trace, config)

// Monitor and log budget utilization
utilizationPct := float64(budgetedTrace.ThinkingTokens) /
    float64(config.ThinkingBudget) * 100
log.Printf("Budget utilization: %.1f%%", utilizationPct)
```

4. Error Handling

```
// Validate configuration
effort, err :=
    llm.ValidateReasoningEffort(config.ReasoningEffort)
if err != nil {
    log.Printf("Invalid effort level: %v", err)
    config.ReasoningEffort = "medium" // fallback
}

// Handle extraction failures gracefully
trace := llm.ExtractReasoningTrace(response, config)
if len(trace.ThinkingContent) == 0 {
    log.Println("Warning: No thinking blocks found")
}
```

5. Performance Monitoring

```
// Track costs over time
thinkingCost, outputCost, totalCost :=
    llm.CalculateReasoningCost(
        trace, config, modelType,
    )

// Log for cost analysis
log.Printf("Request cost: $%.6f (thinking: $%.6f, output: $
    %.6f)",
    totalCost, thinkingCost, outputCost)

// Set up alerts for high costs
if totalCost > 0.10 {
    log.Printf("High cost alert: $%.6f", totalCost)
}
```

6. Prompt Optimization

```
// Use model-appropriate prompts
formattedPrompt := llm.FormatReasoningPrompt(originalPrompt,
    config)

// Customize prompts for specific needs
if needsDetailed {
    config.ReasoningEffort = "high"
    formattedPrompt = llm.FormatReasoningPrompt(originalPrompt,
        config)
}

// Keep prompts focused
// ✓ Good: "Calculate the optimal route between A and B"
// ✗ Bad: "Tell me everything about routing and then
    calculate..."
```

7. Testing and Validation

```
// Test with multiple models
models := []llm.ReasoningModelType{
    llm.ReasoningModelClaude_Sonnet,
    llm.ReasoningModelDeepSeek_R1,
}

for _, modelType := range models {
    config := llm.NewReasoningConfig(modelType)
    // Test reasoning quality and cost
}

// Validate reasoning quality
trace := llm.ExtractReasoningTrace(response, config)
if len(trace.ThinkingContent) < 2 {
    log.Println("Warning: Minimal reasoning detected")
}
```

API Reference

Types

ReasoningConfig

```
type ReasoningConfig struct {  
    Enabled          bool    // Enable reasoning mode  
    ExtractThinking bool    // Extract thinking blocks  
    HideFromUser    bool    // Hide thinking from users  
    ThinkingTags     string // XML tags for thinking  
    ThinkingBudget   int     // Token budget (0=unlimited)  
    ReasoningEffort  string // "low", "medium", "high"  
    ModelType        ReasoningModelType  
}
```

ReasoningTrace

```
type ReasoningTrace struct {  
    ThinkingContent []string // Extracted thinking blocks  
    OutputContent   string   // Final output  
    ThinkingTokens  int      // Tokens used for thinking  
    OutputTokens    int      // Tokens used for output  
    TotalTokens     int      // Total tokens  
}
```

ReasoningModelType

```
const (  
    ReasoningModelOpenAI_01           // OpenAI o1  
    ReasoningModelOpenAI_03           // OpenAI o3  
    ReasoningModelOpenAI_04           // OpenAI o4  
    ReasoningModelClaude_Opus         // Claude Opus  
    ReasoningModelClaude_Sonnet       // Claude Sonnet  
    ReasoningModelDeepSeek_R1         // DeepSeek R1  
    ReasoningModelDeepSeek_Reasoner   // DeepSeek Reasoner  
    ReasoningModelQwQ_32B             // QwQ-32B  
    ReasoningModelGeneric              // Generic reasoning  
)
```

Functions

Configuration Functions

```
// DefaultReasoningConfig returns default configuration
func DefaultReasoningConfig() *ReasoningConfig

// NewReasoningConfig creates model-specific configuration
func NewReasoningConfig(modelType ReasoningModelType)
    *ReasoningConfig

// MergeReasoningConfigs merges two configs (override takes
    precedence)
func MergeReasoningConfigs(base, override *ReasoningConfig)
    *ReasoningConfig

// OptimizeReasoningConfig optimizes config based on context
func OptimizeReasoningConfig(config *ReasoningConfig, ctx
    context.Context) *ReasoningConfig
```

Reasoning Functions

```
// ExtractReasoningTrace extracts thinking blocks from response
func ExtractReasoningTrace(content string, config
    *ReasoningConfig) *ReasoningTrace

// ApplyReasoningBudget enforces token budget constraints
func ApplyReasoningBudget(trace *ReasoningTrace, config
    *ReasoningConfig) *ReasoningTrace

// FormatReasoningPrompt formats prompt for reasoning models
func FormatReasoningPrompt(prompt string, config
    *ReasoningConfig) string
```

Validation Functions

```
// ValidateReasoningEffort validates effort level
func ValidateReasoningEffort(effort string)
    (ReasoningEffortLevel, error)

// IsReasoningModel checks if model name indicates reasoning
    model
func IsReasoningModel(modelName string) (bool,
    ReasoningModelType)
```

Cost Functions

```
// CalculateReasoningCost calculates costs for reasoning
// operation
// Returns (thinkingCost, outputCost, totalCost)
func CalculateReasoningCost(
    trace *ReasoningTrace,
    config *ReasoningConfig,
    modelType ReasoningModelType,
) (float64, float64, float64)

// GetReasoningBudgetRecommendation returns recommended token
// budget
func GetReasoningBudgetRecommendation(useCase string) int
```

Troubleshooting

Issue: No thinking blocks extracted

Cause: Model may not be using the expected thinking tags.

Solution:

```
// Check response format
fmt.Println("Raw response:", response)

// Try different tags
config.ThinkingTags = "thinking,think,thoughts,reasoning"
trace := llm.ExtractReasoningTrace(response, config)
```

Issue: High costs

Cause: Thinking budget too high or wrong model selected.

Solution:

```
// Use cost-effective model
config := llm.NewReasoningConfig(llm.ReasoningModelDeepSeek_R1)

// Set appropriate budget
config.ThinkingBudget =
    llm.GetReasoningBudgetRecommendation("simple")
```

```
// Monitor costs
thinkingCost, outputCost, totalCost :=
    llm.CalculateReasoningCost(trace, config, modelType)
```

Issue: Truncated reasoning

Cause: Budget too restrictive.

Solution:

```
// Increase budget
config.ThinkingBudget =
    llm.GetReasoningBudgetRecommendation("complex")
```

```
// Or use unlimited budget
config.ThinkingBudget = 0
```

Issue: Poor reasoning quality

Cause: Effort level too low or model not suitable.

Solution:

```
// Increase effort level
config.ReasoningEffort = "high"
```

```
// Use more capable model
config = llm.NewReasoningConfig(llm.ReasoningModelClaude_Opus)
```

Additional Resources

- [OpenAI o1 Documentation](#)
- [Claude Extended Thinking](#)
- [DeepSeek R1 Paper](#)
- [HelixCode API Reference](#)
- [Cost Optimization Guide](#)

Support

For questions or issues with reasoning models:

1. Check this documentation
2. Review the API reference
3. Check the [troubleshooting section](#)
4. Open an issue on GitHub

5. Contact HelixCode support

Last Updated: 2025-01-06 **Version:** 1.0.0